

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Aux: Diego Josue Guevara Abaj

Catedrático: Ing. Otto Escobar Leiva

Lab. Arquitectura de Ensambladores y Computadoras 1 B



Práctica: Calculadora de Números Enteros en Ensamblador ARM64

Estudiante: Yesus Rudy Espinoza Pivaral

Carnet: 201020621

Repositorio GitHub:

https://github.com/yesusespinoza201020621/ARQUI1B_2S_P_201020621

Introducción

Aprender ARM64 en bajo nivel (lenguaje ensamblador) es fundamental para los estudiantes de sistemas domina el mundo de la computación moderna y define cómo interactúa el software directamente con el hardware actual.

Actualmente es el estándar de la industria actual

1. Descripción General del programa.

El presente software consiste en una calculadora interactiva de números enteros ejecutada desde la terminal de comandos, desarrollada en su totalidad utilizando lenguaje ensamblador ARM64 puro (AArch64) desde mi sistema Linux Ubuntu. El sistema implementa un menú en consola que permite al usuario seleccionar entre seis operaciones aritméticas diferentes suma, resta, multiplicación, división entera, potencia y radical manejando solamente números enteros los números que sean negativos se mandan una o varias funciones de error como en el caso de la división entre cero que tirar error, manteniendo la ejecución activa hasta que se invoque explícitamente la opción de salida.

2. requisitos técnicos para le ejecución

Para la construcción, compilación, enlace y depuración del sistema se utilizó el siguiente entorno tecnológico:

- **Sistema Operativo:** [Raspberry Pi OS / Ubuntu Linux]
- **Compilador/Ensamblador:** GNU Assembler (as) para AArch64
- **Enlazador:** GNU Linker (ld)
- **Depurador:** GDB (GNU Debugger) / gdb-multiarch
- **Emulador (si aplica):** QEMU para emulación sobre arquitectura x86_64
- **visual studio code(opcional):** pero se maneja de mejor forma los errores se corre la aplicación desde allí o desde la terminal de Ubuntu.

3. Operaciones matemáticas implementadas

las operaciones conocidas de toda la vida

El flujo principal está controlado mediante subrutinas organizadas de forma modular utilizando instrucciones de salto y enlace (bl y ret) en bajo nivel.

Las operaciones son las siguientes:

1. **Suma:** Operación aritmética directa mediante la instrucción `ADD`.
2. **Resta:** Operación aritmética directa mediante la instrucción `SUB`.
3. **Multiplicación:** Operación mediante la instrucción `MUL`.
4. **División Entera:** Cociente de números enteros mediante `SDIV`.
5. **Potencia:** Implementada de manera iterativa utilizando un ciclo basado en multiplicaciones repetidas (`MUL`).
6. **Factorial:** Operación iterativa mediante un bucle de decremento y multiplicación acumulada.

4. validaciones de restricciones

Para garantizar la estabilidad del programa y cumplir con las especificaciones técnicas obligatorias, se desarrollaron bloques de control con saltos condicionales (`cmp`, `b.eq`, `b.lt`, etc.):

- **Menú Interactivo:** Cualquier entrada fuera del rango es rechazada por el programa algún número negativo, opción que no este en el menú alguna letra de mas, se limpia el flujo de entrada y reimprimiendo las opciones del menú.
- **División por Cero:** Antes de realizar la instrucción, se verifica si el registro del divisor es igual a cero. De ser así, se bloquea la operación y se muestra un mensaje de error en consola.
- **Potencia con Exponentes Negativos:** Se valida que el exponente sea mayor o igual a cero antes de iniciar las multiplicaciones iterativas.
- **Factorial de Números Negativos:** Se comprueba que el operando sea un entero no negativo antes de procesar el bucle iterativo.

5. GUÍA DE COMPILACIÓN Y EJECUCIÓN

Para generar el binario ejecutable a partir del archivo de código fuente `.s`, se deben ejecutar los siguientes comandos en la terminal de Linux:

hay 2 formas

se creo el archivo `qemu` para poder ejecutar desde virtual studio code

https://github.com/yesusespinoza201020621/ARQUI1B_2S_P_201020621/blob/main/Makefile.qemu

o desde consola

```
ing-yesusespinoza@ing-yesusespinoza-Latitude-7300:~/Documentos/LabArquit/practicacalcu$ aarch64-linux-gnu-as ARQUI1B_2S_P_201020621.s -o ARQUI1B_2S_P_201020621.o
ing-yesusespinoza@ing-yesusespinoza-Latitude-7300:~/Documentos/LabArquit/practicacalcu$ aarch64-linux-gnu-ld ARQUI1B_2S_P_201020621.o -o ARQUI1B_2S_P_201020621
ing-yesusespinoza@ing-yesusespinoza-Latitude-7300:~/Documentos/LabArquit/practicacalcu$ qemu-aarch64 ./ARQUI1B_2S_P_201020621
Practica Única
Yesus Rudy Espinoza Pivaral 201020621
Calculadora de Numeros enteros en Ensamblador ARM64 desde linux ubuntu
Elige la Operación a Realizar
opcion 1 Suma
Opcion 2 Resta
Opcion 3 multiplicación
Opcion 4 division entera
Opcion 5 potencia
Opcion 6 factorial
█
```

se realizo principalmente desde visual studio code por el mejor manejo de los errores.

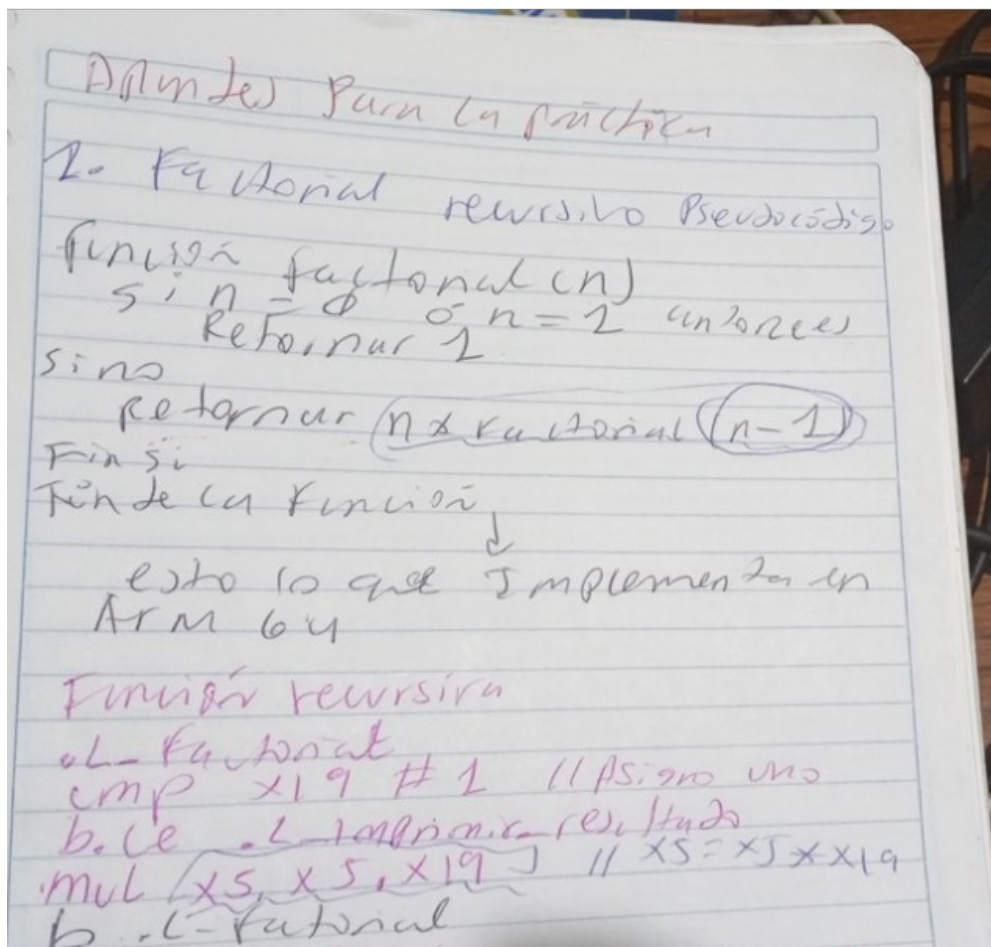
6. código fuente Arm64

esta en el siguiente link en un solo archivo

https://github.com/yesusespinoza201020621/ARQUIB_2S_P_201020621/blob/main/ARQUIB_2S_P_201020621.s

7. capturas de como funciona la calculadora

para empezar y tener Ideas se empezo por pseudocódigo asi para darme una idea y como pasarlo a ensamblador.



Aquí podemos ver por ejemplo la parte de factorial la misma idea se implemento en arm64 hacerlo de forma recursiva.

Ahora vemos como quedaría la implementación final

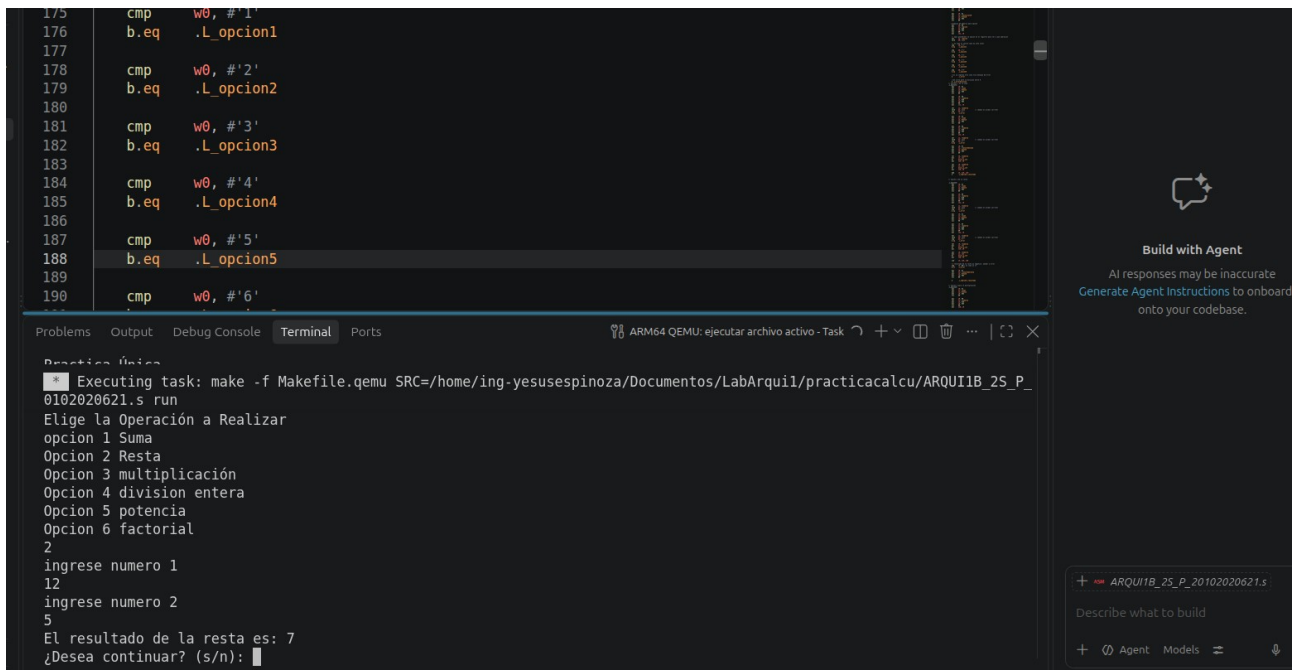
Versión desde la terminal de Linux Ubuntu

```
lng-yesusespinoza@lng-yesusespinoza-Latitude-7300:~/Documentos/LabArqui/practicacalcu$ qemu-aarch64 ./ARQUI1B_2S_P_20102020621
Practica Unica
Vesud Rudy Espinoza Pivaral 201020621
Calculadora de Numeros enteros en Ensamblador ARM64 desde linux ubuntu
Elige la Operación a Realizar
opcion 1 Suma
opcion 2 Resta
opcion 3 multiplicación
opcion 4 division entera
opcion 5 potencia
opcion 6 factorial
1
ingrese numero 1
34
ingrese numero 2
56
El resultado de la suma es: 90
¿Desea continuar? (s/n): s
Elige la Operación a Realizar
opcion 1 Suma
opcion 2 Resta
opcion 3 multiplicación
opcion 4 division entera
opcion 5 potencia
opcion 6 factorial
6
ingrese numero para obtener el factorial
12
El factorial es: 479001600
¿Desea continuar? (s/n):
```

este seria el menú interactivo al decirle que no se sale del programa
acá podemos observar la suma y el factorial

```
Go Run Terminal Help
ARM64 QEMU: ejecutar archivo activo User
Select the task to run
recently used
contributed
grunt
gulp
jake
npm
typescript
python
Show All Tasks...
ARM64 QEMU: ejecutar archivo activo User
116 .L_menu_loop:
162 //espacio de memoria
174 // es como el switch
175 cmp w0, #'1'
176 b.eq .L_opcion1
177
178 cmp w0, #'2'
179 b.eq .L_opcion2
180
181 cmp w0, #'3'
182 b.eq .L_opcion3
183
184 cmp w0, #'4'
185 b.eq .L_opcion4
186
187 cmp w0, #'5'
188 b.eq .L_opcion5
189
190 cmp w0, #'6'
191 b.eq .L_opcion6
192
193 //si se ingresa otra cosa tira mensaje de error
194 b .L_error
195
196 //me sirve para la division entre 0
197 b .L_errordivision
198 // opciónn 1 de la suma
199 .L_opcion1:
200 mov x0, #1
201 ldr x1, =num1
```

así corremos desde visual studio code ya configurado



así sería la operación de resta corrido desde visual studio code

al final igual se corre desde una terminal programa funcional

```
¿Desea continuar? (s/n): s
Elige la Operación a Realizar
opcion 1 Suma
Opcion 2 Resta
Opcion 3 multiplicación
Opcion 4 division entera
Opcion 5 potencia
Opcion 6 factorial
2
ingrese numero 1
7
ingrese numero 2
12
numero, opción o caracter ingreso no valido vuelva a intentar
¿Desea continuar? (s/n):
```

que pasa ti tenemos errores

```
¿Desea continuar? (s/n): s
Elige la Operación a Realizar
opcion 1 Suma
Opcion 2 Resta
Opcion 3 multiplicación
Opcion 4 division entera
Opcion 5 potencia
Opcion 6 factorial
2
ingrese numero 1
7
ingrese numero 2
12
numero, opción o caracter ingreso no valido vuelva a intentar
¿Desea continuar? (s/n):
```

como podemos observar se realiza la operación 7-12 lo cual es -5 el negativo no esta definido por lo cual cuando el restado es menor a cero lo mandamos a error y tira ese mensaje esa función prácticamente es para los errores

para la división es un caso especial cuando se ingreso el segundo numero 0 se manda a ese error

```
¿Desea continuar? (s/n): s
Elige la Operación a Realizar
opcion 1 Suma
opcion 2 Resta
opcion 3 multiplicación
opcion 4 division entera
opcion 5 potencia
opcion 6 factorial
4
ingrese numero 1
34
ingrese numero 2
0
division entre cero no se puede hacer bueno da oo(infinito)
¿Desea continuar? (s/n):
```

en este fragmento de código manejamos el error de la división

```
429 // esto por si es entre cero
430 cmp    x20, #0
431 b.eq   .L_errordivision
```

fuentes bibliográficas

1. Larry D Pyeatt - ARM 64-Bit Assembly Language. L-Newnez (2020) capitulo 3
2. Material de apoyo proporcionada por el auxiliar en su github/PoncheDeFrutas
3. material de apoyo de recursos adicionales de arm64
4. Clase uso Practico de Registros en AArch64